

Blazor Server

Architecture & Security Guide

Patterns, Authentication, and Best Practices

A comprehensive guide covering architecture patterns, security implementations, and Blazor-specific techniques for building robust enterprise applications.

Table of Contents

Part I: Architecture Patterns

- Chapter 1: Understanding Base Classes
- Chapter 2: Entity Framework Configurations
- Chapter 3: The ServiceResult Pattern
- Chapter 4: The Adapter Pattern & Model Agnostic Design
- Chapter 5: Bulk Operations with Entity Framework Core

Part II: Security

- Chapter 6: Role-Based Permissions
- Chapter 7: Password Security
- Chapter 8: Session Management
- Chapter 9: Antiforgery & CSRF Protection
- Chapter 10: Claims-Based Authorization Without ASP.NET Identity

Part III: Blazor-Specific Patterns

- Chapter 11: ProtectedLocalStorage
- Chapter 12: Localization
- Chapter 13: JavaScript Interop

Appendix: Quick Reference

Part I: Architecture Patterns

Chapter 1: Understanding Base Classes

1.1 The Problem: Code Duplication

Look at typical entity classes without inheritance:

```
public class Board
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? UpdatedAt { get; set; }
    // ... other properties
}

public class TaskItem
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? UpdatedAt { get; set; }
    // ... other properties
}

public class Tag
{
    public int Id { get; set; }
    // No audit fields here - inconsistent!
}
```

Problems with this approach:

- **Code duplication** - We repeat Id, CreatedAt, UpdatedAt in every entity
- **Inconsistency** - Some entities have audit fields, some don't
- **Maintenance nightmare** - Adding CreatedById requires modifying every entity

1.2 The Solution: Inheritance

We create **base classes** that contain common properties:

```
// Level 1: Every entity has an ID
public abstract class BaseEntity
{
    public int Id { get; set; }
}

// Level 2: Entities that track who created/modified them
public abstract class BaseAuditableEntity : BaseEntity
{
    public DateTime CreatedAt { get; set; }
    public int? CreatedById { get; set; }
    public DateTime? UpdatedAt { get; set; }
    public int? UpdatedById { get; set; }
}
```

Now entities become cleaner:

```
public class Board : BaseAuditableEntity
{
    public string Name { get; set; }
    public string? Description { get; set; }
    public bool IsArchived { get; set; }
    // Id, CreatedAt, CreatedById, etc. are inherited!
}
```

1.3 Why abstract?

The **abstract** keyword means you cannot create an instance of the class directly. It only exists to be inherited by other classes.

```
// This won't compile
var entity = new BaseEntity();

// This works
var board = new Board();
```

Why? A "BaseEntity" by itself is meaningless. It's not a board, not a task, not a user. It's just a template for other classes.

1.4 Why Two Levels?

Not every entity needs audit fields:

Level 1 - BaseEntity (just Id) - For simple entities like:

- Role - System-defined, no need to track who created them
- Permission - Same as roles
- Join tables like TaskTag - Just relationships, not independent entities

Level 2 - BaseAuditableEntity (Id + audit fields) - For business entities:

- Board, TaskItem, User - Need to know who created/modified them

1.5 The DRY Principle

DRY = Don't Repeat Yourself

```
Before:
Board:      Id, CreatedAt, UpdatedAt, Name, Description, IsArchived
TaskItem:   Id, CreatedAt, UpdatedAt, Title, Description, Priority
Tag:        Id, CreatedAt, UpdatedAt, Name, Color
User:       Id, CreatedAt, UpdatedAt, Email, UserName, PasswordHash

After:
BaseAuditableEntity: Id, CreatedAt, CreatedById, UpdatedAt, UpdatedById
    ■■■ Board:      Name, Description, IsArchived
    ■■■ TaskItem:   Title, Description, Priority
    ■■■ Tag:        Name, Color
    ■■■ User:       Email, UserName, PasswordHash
```

1.6 Real-World Benefit: Adding Features

Imagine adding **soft delete** (mark as deleted instead of actually deleting):

Without base classes: Modify 10+ entity files

With base classes:

```
public abstract class BaseAuditableEntity : BaseEntity
{
    public DateTime CreatedAt { get; set; }
    public int? CreatedById { get; set; }
    public DateTime? UpdatedAt { get; set; }
    public int? UpdatedById { get; set; }

    // NEW: Soft delete - one change, all entities get the feature
}
```

```
    public bool IsDeleted { get; set; }  
    public DateTime? DeletedAt { get; set; }  
    public int? DeletedById { get; set; }  
}
```

This is the power of inheritance: One change affects all inheriting classes.

Chapter 2: Entity Framework Configurations

2.1 Where Do We Define Database Rules?

EF Core needs to know: Which properties are required? What's the maximum length? How are entities related? What columns should be indexed? There are three approaches:

Approach 1: Data Annotations (Simple but Limited)

```
public class User
{
    public int Id { get; set; }

    [Required]
    [MaxLength(256)]
    [EmailAddress]
    public string Email { get; set; }
}
```

Cons: Mixes domain model with database concerns, limited capabilities, clutters entity class.

Approach 2: Fluent API in OnModelCreating (Messy)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>(entity =>
    {
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Email).IsRequired().HasMaxLength(256);
        // ... 20 more lines per entity
    });
    // 10 more entities = 500+ lines!
}
```

Cons: OnModelCreating becomes huge, hard to find configurations.

Approach 3: Separate Configuration Classes (Recommended)

```
// Data/Configurations/Identity/UserConfiguration.cs
public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.HasKey(e => e.Id);

        builder.Property(e => e.Email)
            .IsRequired()
            .HasMaxLength(256);
        builder.HasIndex(e => e.Email).IsUnique();

        builder.HasMany(e => e.UserRoles)
            .WithOne(e => e.User)
            .HasForeignKey(e => e.UserId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

Then in DbContext:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // One line loads ALL configurations from the assembly
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}
```

```
}
```

2.2 Understanding the Fluent API

Primary Keys

```
// Simple primary key
builder.HasKey(e => e.Id);

// Composite primary key (for join tables)
builder.HasKey(e => new { e.UserId, e.RoleId });
```

Property Configuration

```
builder.Property(e => e.Email)
    .IsRequired()           // NOT NULL in database
    .HasMaxLength(256)     // NVARCHAR(256)
    .HasColumnName("email") // Override column name
    .HasDefaultValue("");   // Default value
```

Indexes

```
// Simple index
builder.HasIndex(e => e.Email);

// Unique index (no duplicates allowed)
builder.HasIndex(e => e.Email).IsUnique();

// Composite index
builder.HasIndex(e => new { e.EntityType, e.EntityId });
```

Relationships

```
// One-to-Many: One User has Many UserRoles
builder.HasMany(e => e.UserRoles) // User has many UserRoles
    .WithOne(e => e.User)         // Each UserRole has one User
    .HasForeignKey(e => e.UserId) // Foreign key column
    .OnDelete(DeleteBehavior.Cascade); // Delete UserRoles when User deleted
```

2.3 Delete Behaviors

Behavior	Use Case
Cascade	UserRoles when User is deleted
Restrict	Can't delete Role if users have it
SetNull	AuditLog.UserId when User deleted (keep logs)
NoAction	Database default behavior

2.4 Common Mistakes

```
// BAD - creates NVARCHAR(MAX) column
builder.Property(e => e.Name);

// GOOD - explicit length
builder.Property(e => e.Name).HasMaxLength(100);

// BAD - deleting a role deletes all users with that role
builder.HasMany(e => e.UserRoles)
```



```
.WithOne(e => e.Role)
.OnDelete>DeleteBehavior.Cascade);

// GOOD - can't delete role if users have it
builder.HasMany(e => e.UserRoles)
.WithOne(e => e.Role)
.OnDelete>DeleteBehavior.Restrict);
```

Chapter 3: The ServiceResult Pattern

3.1 The Problem: How Do Services Report Errors?

```
public async Task<User> CreateUserAsync(CreateUserDto dto)
{
    // What if email already exists?
    // What if password is too weak?
    // What if database fails?

    var user = new User { Email = dto.Email };
    await _context.Users.AddAsync(user);
    await _context.SaveChangesAsync();
    return user;
}
```

3.2 Bad Approaches

Return Null

```
public async Task<User?> CreateUserAsync(CreateUserDto dto)
{
    if (await EmailExists(dto.Email))
        return null; // Why null? Email? Validation? Database?
}
```

Problem: Caller doesn't know *why* it failed.

Throw Exceptions

```
public async Task<User> CreateUserAsync(CreateUserDto dto)
{
    if (await EmailExists(dto.Email))
        throw new Exception("Email already exists");
}
```

Problems: Exceptions are for exceptional cases, not business logic. They're expensive and force try/catch everywhere.

3.3 The Solution: ServiceResult Pattern

```
public class ServiceResult<T>
{
    public bool Success { get; private set; }
    public T? Data { get; private set; }
    public string? Message { get; private set; }
    public List<string> Errors { get; private set; } = [];

    public static ServiceResult<T> Ok(T data, string? message = null)
    {
        return new ServiceResult<T>
        {
            Success = true,
            Data = data,
            Message = message
        };
    }

    public static ServiceResult<T> Fail(string error)
    {
        return new ServiceResult<T>
```

```

        {
            Success = false,
            Message = error,
            Errors = [error]
        };
    };
}

public static ServiceResult<T> Fail(List<string> errors)
{
    return new ServiceResult<T>
    {
        Success = false,
        Message = errors.FirstOrDefault(),
        Errors = errors
    };
}
}

```

3.4 Using ServiceResult in Services

```

public async Task<ServiceResult<UserDto>> CreateUserAsync(CreateUserDto dto)
{
    // Validation
    if (await EmailExistsAsync(dto.Email))
        return ServiceResult<UserDto>.Fail("Email already exists");

    var (isStrong, message) = _passwordService.ValidateStrength(dto.Password);
    if (!isStrong)
        return ServiceResult<UserDto>.Fail(message);

    // Create user
    var user = new User
    {
        Email = dto.Email,
        UserName = dto.UserName,
        PasswordHash = _passwordService.Hash(dto.Password)
    };

    await _context.Users.AddAsync(user);
    await _context.SaveChangesAsync();

    // Success
    return ServiceResult<UserDto>.Ok(MapToDto(user), "User created successfully");
}

```

3.5 Using ServiceResult in UI Components

```

@code {
    private async Task HandleSubmit()
    {
        var result = await UserService.CreateUserAsync(_createDto);

        if (result.Success)
        {
            Snackbar.Add(result.Message, Severity.Success);
            NavigationManager.NavigateTo("/users");
        }
        else
        {
            foreach (var error in result.Errors)
            {
                Snackbar.Add(error, Severity.Error);
            }
        }
    }
}

```

```
}  
  }  
}
```

3.6 Benefits

Benefit	Explanation
Explicit	Caller must check Success - can't ignore errors
Informative	Errors contain meaningful messages
Consistent	Same pattern across all services
Type-safe	Compiler enforces handling
No exceptions	Business logic stays clean
Composable	Easy to chain service calls

Step 2: Create the Adapter (The Converter)

Create a class that **adapts** your actual User to the ILmsLearner interface:

```
// Services/LMS/Adapters/SkillCanvasLearnerAdapter.cs
public class SkillCanvasLearnerAdapter : ILmsLearner
{
    private readonly UserDto _user;

    public SkillCanvasLearnerAdapter(UserDto user)
    {
        _user = user;
    }

    // Adapt User properties to ILmsLearner interface
    public int Id => _user.Id;
    public string Name => _user.UserName;           // UserName -> Name
    public string Email => _user.Email;
    public string? AvatarUrl => null;               // User doesn't have avatar
}
```

Step 3: Create a Provider (The Factory)

A provider fetches and adapts users on demand:

```
public class SkillCanvasLearnerProvider : ILmsLearnerProvider
{
    private readonly IUserService _userService;
    private readonly IAuthenticationService _authService;

    public async Task<ILmsLearner?> GetLearnerAsync(int userId)
    {
        var result = await _userService.GetByIdAsync(userId);
        if (!result.Success || result.Data == null)
            return null;

        return new SkillCanvasLearnerAdapter(result.Data);
    }

    public async Task<ILmsLearner?> GetCurrentLearnerAsync()
    {
        var session = await _authService.GetCurrentSessionAsync();
        if (session?.User == null)
            return null;

        return new SkillCanvasLearnerAdapter(session.User);
    }
}
```

Step 4: LMS Uses Only the Interface

Now the LMS **never touches User directly**:

```
public class EnrollmentService
{
    private readonly ILmsLearnerProvider _learnerProvider;

    public async Task<ServiceResult> EnrollCurrentUserAsync(int courseId)
    {
        // Gets an ILmsLearner, not a User!
        var learner = await _learnerProvider.GetCurrentLearnerAsync();
        if (learner == null)
            return ServiceResult.Failure("No authenticated learner");

        var enrollment = new Enrollment
        {
            UserId = learner.Id,           // ILmsLearner.Id
        }
    }
}
```

```

        CourseId = courseId
    };
    // ...
}
}

```

4.4 Why "Model Agnostic"?

Model Agnostic means the LMS doesn't care about your specific models. It works with any user system that provides an adapter.

Scenario	Solution
Today: SkillCanvas with User	SkillCanvasLearnerAdapter
Tomorrow: HR System with Employee	EmployeeLearnerAdapter
Future: Standalone with AppUser	StandaloneUserAdapter

Same LMS code, different adapters.

4.5 Benefits

Benefit	Explanation
Decoupling	LMS doesn't depend on specific user implementation
Reusability	Extract LMS as separate project anytime
Testability	Mock ILmsLearnerProvider in tests
Flexibility	Switch user systems without changing LMS
Single Responsibility	Adapter handles only translation

4.6 When to Use the Adapter Pattern

Use it when:

- You want decoupled modules that can be extracted later
- You need to integrate with multiple user systems
- You want testable code with mockable dependencies
- You're building reusable libraries or modules

Don't use it for:

- Simple internal services that won't be reused
- When tight coupling is acceptable (small projects)

Chapter 5: Bulk Operations with Entity Framework Core

5.1 The Problem: Why Bulk Operations?

When importing large datasets, traditional Entity Framework approaches become painfully slow:

```
// Inserting 1000 articles the "normal" way
foreach (var artigo in artigos)
{
    context.CrmArtigos.Add(artigo);
    await context.SaveChangesAsync(); // 1000 round trips to database!
}
```

Each `SaveChangesAsync()` generates: a network round trip, transaction overhead, identity column retrieval, and change tracking updates. For 1000 records, that's 1000 round trips!

5.2 Approach Comparison

Approach 1: Individual SaveChanges (Slow)

```
// Total time for 100 articles with 4 phases each: ~30 seconds
foreach (var row in excelRows)
{
    var artigo = CreateArtigo(row);
    context.CrmArtigos.Add(artigo);
    await context.SaveChangesAsync(); // Round trip #1

    foreach (var phase in row.Phases)
    {
        var faseArtigo = CreateFaseArtigo(phase, artigo.Id);
        context.CrmFaseArtigos.Add(faseArtigo);
        await context.SaveChangesAsync(); // Round trip #2, #3, #4...
    }
}
```

Approach 2: AddRange + Single SaveChanges (Better)

```
// Add all at once, save once
var artigos = excelRows.Select(r => CreateArtigo(r)).ToList();
context.CrmArtigos.AddRange(artigos);
await context.SaveChangesAsync(); // Single round trip!
```

Cons: Still generates individual INSERT statements. Can't get IDs until `SaveChanges` completes.

Approach 3: EFCore.BulkExtensions (Our Approach)

```
// NuGet: EFCore.BulkExtensions
using EFCore.BulkExtensions;

var bulkConfig = new BulkConfig
{
    PreserveInsertOrder = true,
    SetOutputIdentity = true // KEY: Gets IDs back after insert!
};

await context.BulkInsertAsync(artigos, bulkConfig);
// Now artigos[0].Id, artigos[1].Id, etc. are populated!
```

5.3 Key Concepts

SetOutputIdentity = true

This is the magic that retrieves IDs after bulk insert:

```
var bulkConfig = new BulkConfig
{
    SetOutputIdentity = true // Without this, IDs remain 0!
};

await context.BulkInsertAsync(artigos, bulkConfig);
Console.WriteLine(artigos[0].Id); // 12345 (actual ID from database)
```

PreserveInsertOrder = true

Ensures entities maintain their order, so you can correlate back to source data:

```
var bulkConfig = new BulkConfig
{
    PreserveInsertOrder = true, // artigos[0] stays at index 0
    SetOutputIdentity = true
};

await context.BulkInsertAsync(artigos, bulkConfig);

// Now artigos[i] corresponds to excelRows[i]
for (int i = 0; i < artigos.Count; i++)
{
    var ficha = CreateFicha(excelRows[i], artigos[i].Id);
}
```

5.4 Pattern for Dependent Objects

When you have parent-child relationships, follow this order:

```
// Step 1: Insert parents, get IDs
var bulkConfig = new BulkConfig
{
    PreserveInsertOrder = true,
    SetOutputIdentity = true
};
await context.BulkInsertAsync(artigos, bulkConfig);
// artigos now have IDs!

// Step 2: Create children using parent IDs
var fichas = new List<CrmArtigoFicha>();
for (int i = 0; i < artigos.Count; i++)
{
    fichas.Add(CreateFicha(excelRows[i], artigos[i].Id));
}
await context.BulkInsertAsync(fichas);

// Step 3: For grandchildren, use an index map
var faseArtigos = new List<CrmFaseArtigo>();
var indexMap = new Dictionary<(int artigoId, string phase), int>();

for (int i = 0; i < artigos.Count; i++)
{
    foreach (var phase in excelRows[i].Phases)
    {
        var faseArtigo = new CrmFaseArtigo { Artigoid = artigos[i].Id };
        indexMap[(artigos[i].Id, phase.Name)] = faseArtigos.Count;
        faseArtigos.Add(faseArtigo);
    }
}
```

```
await context.BulkInsertAsync(faseArtigos, bulkConfig);
// faseArtigos now have IDs, use indexMap to find them!
```

5.5 Transaction Handling: All-or-Nothing

```
await using var transaction = await context.Database.BeginTransactionAsync();

try
{
    await context.BulkInsertAsync(artigos, bulkConfig);
    await context.BulkInsertAsync(fichas);
    await context.BulkInsertAsync(faseArtigos, bulkConfig);
    await context.BulkInsertAsync(componentes);

    await transaction.CommitAsync();
}
catch (Exception)
{
    await transaction.RollbackAsync();
    throw; // All changes undone!
}
```

5.6 Performance Comparison

Approach	100 Articles	1000 Articles
Individual SaveChanges	~30 seconds	~5 minutes
AddRange + SaveChanges	~5 seconds	~45 seconds
BulkExtensions	~0.5 seconds	~3 seconds

5.7 Common Mistakes

```
// WRONG: IDs remain 0
await context.BulkInsertAsync(artigos);
Console.WriteLine(artigos[0].Id); // 0!

// RIGHT: IDs are populated
await context.BulkInsertAsync(artigos, new BulkConfig { SetOutputIdentity = true });

// WRONG: Order may change, correlation breaks
await context.BulkInsertAsync(artigos, new BulkConfig { SetOutputIdentity = true });
var ficha = CreateFicha(excelRows[0], artigos[0].Id); // May not match!

// RIGHT: Order preserved
await context.BulkInsertAsync(artigos, new BulkConfig
{
    SetOutputIdentity = true,
    PreserveInsertOrder = true
});

// WRONG: If componentes fails, artigos are already committed!
await context.BulkInsertAsync(artigos, bulkConfig);
await context.BulkInsertAsync(componentes); // Error leaves orphan artigos

// RIGHT: Transaction wraps everything
await using var transaction = await context.Database.BeginTransactionAsync();
// ... all inserts ... then CommitAsync()
```

5.8 When to Use Bulk Operations

Scenario	Recommended Approach
Single entity insert	<code>context.Add() + SaveChangesAsync()</code>
5-50 entities	<code>AddRange() + SaveChangesAsync()</code>
50+ entities	<code>BulkInsertAsync()</code>
With dependent objects	<code>BulkInsertAsync()</code> with <code>SetOutputIdentity</code>
Upsert (insert or update)	<code>BulkInsertOrUpdateAsync()</code>

Part II: Security

Chapter 6: Role-Based Permissions

6.1 The Bad Way: Boolean Flags

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }

    // Authorization via booleans
    public bool IsAdmin { get; set; }
    public bool CanCreateBoards { get; set; }
    public bool CanDeleteBoards { get; set; }
    public bool CanManageUsers { get; set; }
    public bool CanViewReports { get; set; }
    // ... 20 more boolean flags
}
```

Why this is bad:

- **Database bloat** - 20 permissions = 20 columns per user
- **No grouping** - 100 managers × 20 flags to maintain
- **Hard to change** - New permission = column + property + forms + checks
- **No hierarchy** - No relationship between Admin and Manager

6.2 The Good Way: Role-Permission Model

User ■■→ UserRole ■■→ Role ■■→ RolePermission ■■→ Permission

Users have Roles:

Luis → [User, Manager]

Ana → [User]

João → [User, Admin]

Roles have Permissions:

User → [Boards.View, Tasks.Create, Tasks.Edit]

Manager → [Boards.Create, Boards.Edit, Users.View]

Admin → [Users.Create, Users.Delete, System.ViewAuditLogs]

Result:

Luis can: Boards.View, Tasks.Create, Tasks.Edit (from User)
+ Boards.Create, Boards.Edit, Users.View (from Manager)

6.3 Benefits

Single Source of Truth: Change a role once, all users updated.

"Managers can now delete tasks."

- Old way: Update 100 user records

- New way: Add Tasks.Delete to Manager role (1 change)

Hierarchical Levels:

```
public class Role
{
    public int HierarchyLevel { get; set; }
    // 0 = User, 1 = Manager, 2 = Admin, 3 = SuperAdmin
}

// Can this user manage that user?
bool canManage = currentUser.HierarchyLevel > targetUser.HierarchyLevel;
```

6.4 Permission Naming Convention

```
public static class Permissions
{
    public static class Boards
    {
        public const string View = "Boards.View";
        public const string Create = "Boards.Create";
        public const string Edit = "Boards.Edit";
        public const string Delete = "Boards.Delete";
    }
}

// BAD - typo won't be caught until runtime
if (HasPermission(userId, "Bords.Create")) // Typo!

// GOOD - compiler catches errors
if (HasPermission(userId, Permissions.Boards.Create))
```

Chapter 7: Password Security

7.1 Why We Never Store Plain Passwords

Scenario: Your database gets hacked (it happens to the best companies).

Storage Method	Hacker sees	Result
Plain text	password123	Immediate access to all accounts
Hashed	\$2a\$12\$LQv3c1yq...	Useless without cracking

Rule #1: Never, ever store plain text passwords.

7.2 What is Hashing?

Hashing is a **one-way function**: Input → Output, but you cannot reverse it.

Password → [Hash Function] → Hash
"abc123" → [BCrypt] → "\$2a\$12\$R9h/cIPz0gi.URNNX3kh20..."

Hash → [???] → Password
"\$2a\$12\$R9h/cIPz0gi.URNNX3kh20..." → ??? → IMPOSSIBLE

7.3 Why BCrypt (Not MD5 or SHA)

MD5 and SHA are **designed to be fast**. A modern GPU can compute 180 billion MD5 hashes per second. BCrypt is **intentionally slow** with a configurable work factor.

Work Factor	Time per Hash	Hashes/Second
10	~100ms	10
12	~300ms	3
14	~1 second	1

8-character password cracking time:
MD5: 6.6 quadrillion ÷ 180 billion = ~10 hours
BCrypt: 6.6 quadrillion ÷ 3 = 70 million years

7.4 What is Salting?

A **salt** is a random string added to the password before hashing. Same password + different salts = different hashes.

Without salt:
"password123" → [Hash] → "5f4dcc3b5aa..."
Two users with same password = same hash (bad!)

With salt:
"password123" + "x7Kj9L" → [Hash] → "a1b2c3..."
"password123" + "mN4pQ2" → [Hash] → "d4e5f6..."
Same password, different hashes (good!)

BCrypt includes the salt in the hash itself:
\$2a\$12\$LQv3c1yqT.SiVHn4FZNVCuqQ.aXP1vF...

- ■ ■ ■ ■ Salt + Hash (combined)
- ■ ■ ■ ■ Work factor (12)
- ■ ■ ■ ■ BCrypt version
- ■ ■ ■ ■ Algorithm identifier

7.5 Implementation

```
public class PasswordService : IPasswordService
{
    private const int WorkFactor = 12;

    public string Hash(string password)
    {
        // BCrypt automatically generates a random salt
        return BCrypt.Net.BCrypt.HashPassword(password, WorkFactor);
    }

    public bool Verify(string password, string hash)
    {
        // BCrypt extracts the salt from the stored hash
        return BCrypt.Net.BCrypt.Verify(password, hash);
    }
}
```

7.6 Account Lockout

```
if (user.LockoutEnd > DateTime.UtcNow)
    return LoginResult.Locked(user.LockoutEnd.Value);

if (!_passwordService.Verify(password, user.PasswordHash))
{
    user.FailedLoginAttempts++;

    // Lock after 5 failed attempts
    if (user.FailedLoginAttempts >= 5)
        user.LockoutEnd = DateTime.UtcNow.AddMinutes(15);

    return LoginResult.InvalidCredentials(5 - user.FailedLoginAttempts);
}

// Success - reset attempts
user.FailedLoginAttempts = 0;
user.LockoutEnd = null;
```

7.7 The "Invalid Credentials" Message

```
// BAD - reveals that email exists
if (user == null) return "Email not found";
if (!validPassword) return "Incorrect password";

// GOOD - doesn't reveal anything
if (user == null || !validPassword)
    return "Invalid credentials";
```

Why? An attacker could use "Email not found" to build a list of valid emails.

Chapter 8: Session Management

8.1 The Problem: HTTP is Stateless

Request 1: "Hi, I'm Luis, here's my password" → Server: "OK, logged in"
Request 2: "Show me my boards" → Server: "Who are you?"

Solution: Sessions

Login: "Here's my password" → Server: "OK, here's token: abc123"

Request 2: "Show boards (token: abc123)" → Server: "Ah, Luis! Here you go"

8.2 How Session Tokens Work

Step 1: Generate Token on Login

```
var token = GenerateSecureToken(); // "x7Kj9LmN4pQ2..."

var session = new UserSession
{
    UserId = user.Id,
    TokenHash = HashToken(token), // Store hash, not plain token
    CreatedAt = DateTime.UtcNow,
    ExpiresAt = DateTime.UtcNow.AddHours(24),
    IPAddress = GetClientIp(),
    UserAgent = GetUserAgent()
};

await _context.UserSessions.AddAsync(session);
return LoginResult.Success(user, token); // Return plain token once
```

Step 2: Store Token in Browser Cookie

```
Response.Cookies.Append("session_token", token, new CookieOptions
{
    HttpOnly = true, // JavaScript can't access (XSS protection)
    Secure = true, // Only sent over HTTPS
    SameSite = SameSiteMode.Strict, // Only same-origin requests
    Expires = DateTime.UtcNow.AddHours(24)
});
```

Step 3: Validate Token on Each Request

```
public async Task<User?> ValidateSessionAsync(string token)
{
    var tokenHash = HashToken(token);

    var session = await _context.UserSessions
        .Include(s => s.User)
        .FirstOrDefaultAsync(s =>
            s.TokenHash == tokenHash &&
            s.ExpiresAt > DateTime.UtcNow &&
            !s.IsRevoked);

    if (session == null) return null;

    session.LastActivityAt = DateTime.UtcNow;
    await _context.SaveChangesAsync();
    return session.User;
}
```

8.3 Cookie Security Options

Option	Attack Prevented
HttpOnly	XSS (cross-site scripting)
Secure	Man-in-the-middle
SameSite=Strict	CSRF (cross-site request forgery)

8.4 Multiple Sessions (Multi-Device Support)

Luis's sessions:

- Session 1: Work PC (Chrome, IP: 192.168.1.100)
- Session 2: Home Laptop (Firefox, IP: 85.241.52.18)
- Session 3: Phone (Safari, IP: 10.0.0.45)

```
// Log out all other sessions (after password change)
public async Task RevokeAllSessionsExceptAsync(int userId, int currentSessionId)
{
    var sessions = await _context.UserSessions
        .Where(s => s.UserId == userId && s.Id != currentSessionId)
        .ToListAsync();

    foreach (var session in sessions)
        session.IsRevoked = true;

    await _context.SaveChangesAsync();
}
```

Safe to ignore:

- Read-only operations (GET requests)
- Non-sensitive preferences (language, theme)
- Public endpoints (no authentication required)

Never ignore:

- Financial transactions
- Password changes
- Account deletion
- Data modification

Chapter 10: Claims-Based Authorization

Key Insight: [Authorize], <AuthorizeView>, and IsInRole() work with **Claims**, not with ASP.NET Identity. Identity is just one way to create claims - you can build your own!

10.1 What Are Claims?

A **Claim** is a statement about a user - a key-value pair that describes something about them.

```
// Examples of claims:
new Claim(ClaimTypes.Name, "john.doe")           // User's name
new Claim(ClaimTypes.Email, "john@example.com")  // User's email
new Claim(ClaimTypes.Role, "Admin")              // User is in Admin role
new Claim("permission", "users.create")          // Custom claim: has permission
new Claim("department", "Engineering")           // Custom claim: department
```

ClaimsPrincipal Structure

```
ClaimsPrincipal (represents the user)
    ■■■ ClaimsIdentity (authentication method, e.g., "SkillCanvasAuth")
        ■■■ Claims[] (list of key-value pairs)
            ■■■ ClaimTypes.NameIdentifier = "123"
            ■■■ ClaimTypes.Name = "john.doe"
            ■■■ ClaimTypes.Role = "Admin"
            ■■■ ClaimTypes.Role = "Manager" ← Multiple role claims!
            ■■■ "permission" = "users.create"
```

Key Point: A user can have multiple claims of the same type - this is how we support multiple roles!

10.2 Claims vs ASP.NET Identity

Many developers think you need ASP.NET Identity for authorization. **This is a misconception.** The authorization system only cares about Claims - it doesn't know or care where they came from.

Feature	ASP.NET Identity	Custom Claims System
User storage	Identity tables	Your own User table
Password hashing	Built-in	You implement (BCrypt)
[Authorize] works?	Yes	Yes!
<AuthorizeView> works?	Yes	Yes!
IsInRole() works?	Yes	Yes!

10.3 How Authorization Actually Works

```
// When you write:
[Authorize(Roles = "Admin,Manager")]

// The framework checks if ClaimsPrincipal has a claim where:
// - Type == ClaimTypes.Role
// - Value == "Admin" OR Value == "Manager"

// IsInRole() is equivalent to:
user.IsInRole("Admin")
// Same as:
user.Claims.Any(c => c.Type == ClaimTypes.Role && c.Value == "Admin")
```

10.4 Building a Custom AuthenticationStateProvider

```
public class CustomAuthStateProvider : AuthenticationStateProvider
{
    private readonly IAuthenticationService _authService;
    private AuthenticationState? _cachedState;

    public override async Task<AuthenticationState> GetAuthenticationStateAsync()
    {
        // Return cached state if available (prevents race conditions)
        if (_cachedState != null) return _cachedState;

        try
        {
            var result = await _authService.GetCurrentUserAsync();

            if (result.Success && result.Data != null)
            {
                // BUILD CLAIMS FROM YOUR USER MODEL
                var claims = BuildClaims(result.Data);
                var identity = new ClaimsIdentity(claims, "MyAuthType");
                var principal = new ClaimsPrincipal(identity);

                _cachedState = new AuthenticationState(principal);
            }
            else
            {
                _cachedState = new AuthenticationState(new ClaimsPrincipal());
            }
        }
        catch
        {
            _cachedState = new AuthenticationState(new ClaimsPrincipal());
        }

        return _cachedState;
    }

    // Call after login/logout to refresh state
    public void NotifyAuthStateChanged()
    {
        _cachedState = null;
        NotifyAuthenticationStateChanged(GetAuthenticationStateAsync());
    }
}
```

10.5 Mapping Your User Model to Claims

```
private List<Claim> BuildClaims(UserDto user)
{
    var claims = new List<Claim>
    {
        new(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new(ClaimTypes.Name, user.UserName),
        new(ClaimTypes.Email, user.Email),
        new("display_name", user.DisplayName ?? user.UserName)
    };

    // Add role claims - ONE CLAIM PER ROLE
    foreach (var role in user.Roles)
    {
        claims.Add(new Claim(ClaimTypes.Role, role));
    }
}
```

```

        // Add permission claims
        foreach (var permission in user.Permissions)
        {
            claims.Add(new Claim("permission", permission));
        }

        return claims;
    }
}

```

10.6 Using Authorization in Blazor

```

// Page-level protection
[attribute [Authorize] // Must be logged in
[attribute [Authorize(Roles = "Admin,SuperAdmin")]] // Specific roles

// Conditional UI
<AuthorizeView>
    <Authorized>
        <MudText>Welcome, @context.User.Identity?.Name!</MudText>
    </Authorized>
    <NotAuthorized>
        <MudButton Href="/login">Login</MudButton>
    </NotAuthorized>
</AuthorizeView>

// Role-specific content
<AuthorizeView Roles="Admin,SuperAdmin">
    <Authorized>
        <MudNavLink Href="/admin/users">User Management</MudNavLink>
    </Authorized>
</AuthorizeView>

```

10.7 Permission-Based Authorization

```

// Extension method for fine-grained control
public static class ClaimsPrincipalExtensions
{
    public static bool HasPermission(this ClaimsPrincipal? user, string permission)
    {
        if (user == null) return false;
        return user.Claims.Any(c =>
            c.Type == "permission" &&
            c.Value.Equals(permission, StringComparison.OrdinalIgnoreCase));
    }
}

// Usage in components
<AuthorizeView>
    <Authorized>
        @if (context.User.HasPermission("users.create"))
        {
            <MudButton OnClick="CreateUser">Create User</MudButton>
        }
        @if (context.User.HasPermission("users.delete"))
        {
            <MudButton Color="Color.Error">Delete User</MudButton>
        }
    </Authorized>
</AuthorizeView>

```

10.8 Program.cs Registration

```

// Register your custom provider

```

```
builder.Services.AddScoped<CustomAuthStateProvider>();  
builder.Services.AddScoped<AuthenticationStateProvider>(sp =>  
    sp.GetRequiredService<CustomAuthStateProvider>());  
  
// Required for [Authorize] and <AuthorizeView>  
builder.Services.AddAuthorizationCore();
```


Part III: Blazor-Specific Patterns

Chapter 11: ProtectedLocalStorage

11.1 What is ProtectedLocalStorage?

ProtectedLocalStorage is a Blazor Server service that provides secure, encrypted access to the browser's localStorage. Unlike regular JavaScript localStorage, it encrypts data before storing and protects against tampering.

```
@inject ProtectedLocalStorage LocalStorage

// Store a value (encrypted automatically)
await LocalStorage.SetAsync("user_preference", "dark_mode");

// Retrieve a value (decrypted automatically)
var result = await LocalStorage.GetAsync<string>("user_preference");
if (result.Success)
{
    var value = result.Value; // "dark_mode"
}
```

11.2 The Prerendering Problem

Blazor Server uses **prerendering** by default. The page is first rendered on the server (no browser exists yet), then sent to the client, then Blazor reconnects via SignalR. During server prerendering, there is **no browser**—so ProtectedLocalStorage throws an exception!

```
// BAD - This will fail during prerendering!
protected override async Task OnInitializedAsync()
{
    var result = await LocalStorage.GetAsync<string>("token"); // EXCEPTION!
}

// GOOD - OnAfterRenderAsync runs after browser is available
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        var result = await LocalStorage.GetAsync<string>("token"); // Works!
        if (result.Success)
        {
            _token = result.Value;
            StateHasChanged(); // Re-render with loaded data
        }
    }
}
```

11.3 UI Pattern: Loading State

```
@if (!IsAuthLoaded)
{
    <MudSkeleton Width="200px" Height="32px" />
    <MudProgressLinear Indeterminate="true" />
    return;
}

@* Real content here - only shown after auth loads *@
<MudText>Welcome, @CurrentUser.DisplayName!</MudText>
```

11.4 Common Mistakes

```

// WRONG - can throw if storage unavailable
var result = await LocalStorage.GetAsync<string>("key");
var value = result.Value; // May be null!

// CORRECT - always check Success
if (result.Success)
{
    var value = result.Value;
}

// WRONG - UI won't update after loading data
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        _data = await LoadDataAsync();
        // UI still shows old state!
    }
}

// CORRECT - call StateHasChanged()
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        _data = await LoadDataAsync();
        StateHasChanged(); // UI updates!
    }
}

```

Chapter 12: Localization

12.1 Overview

Localization allows your app to display text in different languages using resource files (.resx), `IStringLocalizer`, and a `CultureController` for language switching.

12.2 Resource Files

```
Project/
■■■■ SharedResources.cs          # Marker class (PROJECT ROOT!)
■■■■ Resources/
    ■■■■ SharedResources.resx      # Default (English)
    ■■■■ SharedResources.pt-PT.resx # Portuguese

SharedResources.resx:
<data name="Login"><value>Login</value></data>
<data name="Logout"><value>Logout</value></data>

SharedResources.pt-PT.resx:
<data name="Login"><value>Entrar</value></data>
<data name="Logout"><value>Sair</value></data>
```

CRITICAL: The marker class file must be in the project root, not inside the Resources folder. `IStringLocalizer` uses the file location to resolve resources.

12.3 The Marker Class

```
namespace SkillCanvas; // ROOT namespace, NOT SkillCanvas.Resources!

/// <summary>
/// Marker class for localization resources.
/// </summary>
public class SharedResources
{
    // Empty - just a marker for IStringLocalizer<SharedResources>
}
```

12.4 Using `IStringLocalizer` in Components

```
@inject IStringLocalizer<SharedResources> L

<MudButton>@L["Login"]</MudButton>
<MudNavLink>@L["Home"]</MudNavLink>
<MudText>@L["Welcome"]</MudText>
```

12.5 `CultureController` for Language Switching

```
[Route("[controller]/[action]")]
public class CultureController : Controller
{
    [IgnoreAntiforgeryToken] // Required for Blazor navigation
    public IActionResult Set(string culture, string redirectUri)
    {
        if (!string.IsNullOrEmpty(culture))
        {
            HttpContext.Response.Cookies.Append(
                CookieRequestCultureProvider.DefaultCookieName,
                CookieRequestCultureProvider.MakeCookieValue(
```

```

        new RequestCulture(culture, culture)),
    new CookieOptions
    {
        Expires = DateTimeOffset.UtcNow.AddYears(1),
        IsEssential = true
    }
    );
}
return LocalRedirect(redirectUri ?? "/");
}
}

```

12.6 LanguageSwitcher Component

```

@Inject NavigationManager NavigationManager

<MudMenu Icon="@Icons.Material.Filled.Language">
    <MudMenuItem OnClick="@(() => SetCulture("en"))">English</MudMenuItem>
    <MudMenuItem OnClick="@(() => SetCulture("pt-PT"))">Português</MudMenuItem>
</MudMenu>

@code {
    private void SetCulture(string culture)
    {
        var uri = new Uri(NavigationManager.Uri)
            .GetComponents(UriComponents.PathAndQuery, UriFormat.Unescaped);

        NavigationManager.NavigateTo(
            $"Culture/Set?culture={Uri.EscapeDataString(culture)}" +
            $"&redirectUri={Uri.EscapeDataString(uri)}",
            forceLoad: true); // Full page reload required!
    }
}

```

12.7 Program.cs Configuration

```

// Add localization services
builder.Services.AddLocalization(options => options.ResourcesPath = "Resources");
builder.Services.AddControllers();

// Configure middleware
var supportedCultures = new[] { "en", "pt-PT" };
var localizationOptions = new RequestLocalizationOptions()
    .SetDefaultCulture("pt-PT")
    .AddSupportedCultures(supportedCultures)
    .AddSupportedUICultures(supportedCultures);

localizationOptions.RequestCultureProviders = new List<IRequestCultureProvider>
{
    new CookieRequestCultureProvider(), // Read from cookie first
    new QueryStringRequestCultureProvider()
};

app.UseRequestLocalization(localizationOptions);
app.MapControllers();

```

Chapter 13: JavaScript Interop

Key Insight: JavaScript interop in Blazor Server is NOT available during prerendering. You must handle `InvalidOperationException` gracefully and defer JS calls to `OnAfterRenderAsync`.

13.1 What is JavaScript Interop?

Blazor runs C# code, but sometimes you need to interact with browser JavaScript APIs (DOM manipulation, `localStorage`, third-party JS libraries). **JavaScript Interop (JS Interop)** is the bridge between C# and JavaScript.

```
// Calling JavaScript from C#
@Inject IJSRuntime JS

var result = await JS.InvokeAsync<string>("localStorage.getItem", "myKey");

// Or call a custom function
await JS.InvokeVoidAsync("myJsFunction", arg1, arg2);

// JavaScript side
window.myJsFunction = function(arg1, arg2) {
    console.log("Called from C#!", arg1, arg2);
};
```

13.2 The Blazor Server Rendering Timeline

Phase 1: PRERENDER (Server-Side, NO JavaScript)

- Initial HTTP request arrives at server
- Components render to static HTML
- NO SignalR connection yet
- ■ JS Interop calls WILL FAIL

Phase 2: HTML SENT TO BROWSER

- Static HTML displayed to user (fast first paint!)
- Blazor JavaScript bundle loads
- SignalR connection establishing...
- ■ JS Interop still not ready

Phase 3: INTERACTIVE MODE (SignalR Connected)

- SignalR WebSocket connection established
- `OnAfterRenderAsync(firstRender: true)` fires
- ■ JS Interop NOW AVAILABLE

13.3 Why JS Interop Fails During Prerender

```
// This FAILS during prerender!
protected override async Task OnInitializedAsync()
{
    var value = await JS.InvokeAsync<string>("localStorage.getItem", "key");
}

// Error:
// InvalidOperationException: JavaScript interop calls cannot be issued at
// this time. This is because the component is being statically rendered.
```

13.4 Safe Patterns for JS Interop

Pattern 1: OnAfterRenderAsync (Recommended)

```
@code {
    private string? _storedValue;
    private bool _isLoading = false;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            // ■ SAFE - SignalR is connected, JS is available
            _storedValue = await JS.InvokeAsync<string>("localStorage.getItem", "key");
            _isLoading = true;
            StateHasChanged(); // Re-render with loaded data
        }
    }
}
```

Pattern 2: Try-Catch with Graceful Fallback

```
public async Task<string?> GetSessionTokenAsync()
{
    try
    {
        var result = await _localStorage.GetAsync<string>("session_token");
        return result.Success ? result.Value : null;
    }
    catch (InvalidOperationException)
    {
        // Expected during prerender - JS not available
        _logger.LogDebug("Session token unavailable (prerender phase)");
        return null;
    }
    catch (Exception ex)
    {
        // Actual errors should be logged as ERROR
        _logger.LogError(ex, "Error retrieving session token");
        return null;
    }
}
```

13.5 Common Pitfalls and Solutions

```
// ■ WRONG - OnInitializedAsync runs during prerender
protected override async Task OnInitializedAsync()
{
    _value = await JS.InvokeAsync<string>("localStorage.getItem", "key");
}

// ■ CORRECT - OnAfterRenderAsync is safe
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        _value = await JS.InvokeAsync<string>("localStorage.getItem", "key");
        StateHasChanged();
    }
}

// ■ WRONG - Infinite loop!
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    _value = await LoadDataAsync(); // Always loads
    StateHasChanged(); // Re-renders → OnAfterRenderAsync → repeat!
```

```

}

// ■ CORRECT - Only load once
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender) // Only on first render!
    {
        _value = await LoadDataAsync();
        StateHasChanged();
    }
}

```

13.6 Handling Null State in UI

```

@* ■ WRONG - Throws if _user is null during prerender *@
<span>Welcome, @_user.DisplayName!</span>

@* ■ CORRECT - Handle null state *@
@if (_user != null)
{
    <span>Welcome, @_user.DisplayName!</span>
}
else
{
    <MudSkeleton Width="100px" />
}

```

13.7 Exception Handling Best Practices

Scenario	Exception	Log Level
Prerender phase	InvalidOperationException	Debug
Language change	InvalidOperationException	Debug
Network error	JSException	Error
Deserialization failure	JsonException	Error

13.8 Key Takeaways

- **JS Interop requires SignalR** - Not available during prerender
- **Use OnAfterRenderAsync** - The ONLY safe lifecycle method for JS calls
- **Check firstRender** - Avoid infinite render loops
- **Handle InvalidOperationException** - Expected during prerender, log as Debug
- **Show loading states** - UI must handle null/loading gracefully
- **Cache when possible** - Prevent concurrent JS calls

Appendix: Quick Reference

Security Checklist

Area	Best Practice
Passwords	BCrypt with work factor 12+, never store plain
Sessions	Hash tokens, HttpOnly + Secure + SameSite cookies
CSRF	[ValidateAntiForgeryToken] on state-changing endpoints
Errors	Generic messages ("Invalid credentials")
Lockout	5 failed attempts → 15 min lockout
Claims	Build ClaimsPrincipal from your user model

Architecture Patterns

Pattern	When to Use
Base Classes	Share common properties (Id, audit fields)
ServiceResult	Return success/failure from service methods
IEntityTypeConfiguration	Separate EF config per entity
Adapter Pattern	Decouple modules, enable model-agnostic design
Bulk Operations	Import 50+ entities with SetOutputIdentity
Role-Permission Model	Flexible authorization

Blazor Server Lifecycle

Method	When It Runs	Use For
OnInitializedAsync	Component init (pre-render + interactive)	Fetch data from services
OnAfterRenderAsync	After DOM render (browser available)	ProtectedLocalStorage, JS interop
OnParametersSetAsync	When parameters change	React to parameter updates

EF Core Bulk Operations

Config Option	Purpose
SetOutputIdentity = true	Get auto-generated IDs back after insert
PreserveInsertOrder = true	Maintain correlation with source data
UpdateByProperties	Specify columns for upsert matching

Claims Authorization

Claim Type	Purpose
ClaimTypes.NameIdentifier	User ID
ClaimTypes.Name	Username
ClaimTypes.Role	Role membership (one per role)
"permission"	Fine-grained permissions

This guide compiles essential patterns and security practices for building robust Blazor Server applications. Apply these consistently across your projects for maintainable, secure code.